

Gestión de Estudiantes: Persistencia de Datos en Java con JDBC, SQLite y el Patrón DAO

Cristian Camilo Girata Ravelo

Ingeniera De Software

Universidad Manuela Beltrán

Juan José Osorio Tabares

Taller de Programación

Bogotá D.C

25/10/2026

Contenido

Introducción 4

Objetivos 5

Marco Conceptual 5

Descripción Sobre La Práctica 8

Requerimientos Funcionales y Técnicos 9

Tecnologías Utilizadas 10

Diseño de la Arquitectura 11

Diseño de la Base de Datos 14

Implementación de JDBC 16

Desarrollo de las Operaciones CRUD 17

Validaciones y Manejo de Excepciones 20

Pruebas Realizadas y Resultados Comprobados 21

Evidencias de Funcionamiento 24

Instrucciones de Instalación y Ejecución 29

Enlaces de la Entrega 30

Conclusiones 30

Referencias 32

Gestión de Estudiantes: Persistencia de Datos en Java con JDBC, SQLite y el Patrón DAO

Casi todas las aplicaciones de software necesitan conservar información más allá del tiempo en que el programa está en ejecución. Mientras los objetos de Java residen en la memoria y desaparecen al cerrar la aplicación, la persistencia de datos permite guardarlos en un medio persistente, como una base de datos, para recuperarlos posteriormente. En Java, la forma estándar de comunicarse con una base de datos relacional es la API JDBC, que permite establecer conexiones, ejecutar sentencias SQL y procesar sus resultados (Oracle, s.f.-b).

Programar bien esta comunicación no es trivial. En un estudio sobre 423 errores de acceso a base de datos en siete aplicaciones Java de código abierto se encuentra que los principales problemas están en las consultas SQL, el esquema y el manejo de la API. En el caso de JDBC los más comunes son los relacionados con las consultas (Liu et al., 2024). Además existe un tipo de ataque muy estudiado llamado inyección SQL donde se introduce código malicioso en las consultas para acceder o modificar información de forma no autorizada (Nasereddin et al., 2021). Este trabajo responde a la Práctica de Persistencia de Datos en Java de la asignatura Taller de Programación. Se desarrolló una aplicación de gestión de estudiantes que almacena la información en SQLite, un motor de base de datos embebido que guarda toda la base en un solo archivo y no requiere un servidor (Gaffney et al., 2022). El documento describe los objetivos, el marco conceptual, los requerimientos, la arquitectura, el diseño de la base de datos, la implementación con JDBC, las validaciones, las pruebas realizadas con sus resultados observados y las instrucciones para instalar y ejecutar el proyecto.

Objetivos

Objetivo General

Desarrollar una aplicación de consola en Java que gestione la información de estudiantes y la almacene de forma persistente en una base de datos SQLite, aplicando JDBC, consultas parametrizadas y el patrón de diseño DAO.

Objetivos Específicos

1. Diseñar una base de datos relacional con restricciones de integridad para almacenar la información de los estudiantes.
2. Implementar la conexión con la base de datos mediante JDBC y la creación automática del esquema al iniciar la aplicación.
3. Implementamos las operaciones CRUD (Create, Read, Update, Delete) utilizando PreparedStatement, para evitar inyección SQL.
4. Los códigos están estructurados en capas (presentación, servicio, acceso a datos), siguiendo el patrón DAO.
5. Se llevan a cabo validaciones de datos de entrada y control de excepciones. Esto permite mostrar mensajes comprensibles sin afectar al correcto funcionamiento de la aplicación.
6. Se comprueba el buen funcionamiento de la aplicación y persistencia de datos. Para ello se han realizado pruebas automáticas y pruebas manuales (documentadas).

Marco Conceptual

Persistencia de Datos

La persistencia es la capacidad de un sistema para conservar el estado de sus datos después de que termina el proceso que los creó. En una aplicación orientada a objetos, los objetos existen en la memoria de la máquina virtual de Java; para que sobrevivan al cierre del programa deben convertirse en registros de un almacenamiento permanente, como un archivo o una base de datos relacional, y reconstruirse cuando se necesiten. En este proyecto cada objeto `Estudiante` corresponde a una fila de la tabla `estudiantes`.

Java Database Connectivity (JDBC)

JDBC es la API estándar de Java para acceder a bases de datos relacionales. Permite conectarse a una fuente de datos, enviar consultas y sentencias de actualización y procesar los resultados que devuelve la base de datos (Oracle, s.f.-b). Sus elementos principales son `Connection`, que representa la sesión con la base de datos; `PreparedStatement`, que contiene una sentencia SQL precompilada con parámetros, y `ResultSet`, que recorre las filas obtenidas. Cada motor de base de datos aporta un controlador que implementa estas interfaces; en este proyecto se usa sqlite-jdbc, una biblioteca que permite crear y consultar archivos SQLite desde Java e incluye las bibliotecas nativas para Windows, macOS y Linux en un solo archivo JAR (Xerial, s.f.).

SQLite como Base de Datos Embebida

SQLite es un motor de base de datos relacional que se ejecuta dentro del mismo proceso de la aplicación y guarda la base de datos completa en un único archivo. Gaffney et al. (2022) señalan que es uno de los motores de bases de datos más desplegados y que está orientado principalmente a cargas transaccionales. Su propia documentación explica que no compite con

En este caso puede ser usado en entornos cliente-servidor, pero más bien está orientado hacia el almacenamiento basado en archivos. Además es recomendable para aplicaciones locales y también para entornos educativos. El motivo es que no necesita de instalación o administración (SQLite, 2025).

Operaciones CRUD

CRUD es un acrónimo que representa las cuatro operaciones básicas sobre datos persistentes. Crear (create), Leer (read), Actualizar (update) y Eliminar (delete). En SQL se traducen en las instrucciones INSERT, SELECT, UPDATE y DELETE. En JDBC las sentencias que modifican datos (INSERT, UPDATE, DELETE) se procesan con executeUpdate() que devuelve el número de filas afectadas. Las consultas (SELECT) se procesan con executeQuery() que devuelve un ResultSet. (Oracle, s.f.-b)

Patrón Data Access Object (DAO)

El patrón DAO propone que toda la parte de acceso a datos esté encapsulada en un objeto. De este modo se logra independizar la lógica de negocio de la API de almacenamiento. Los beneficios principales son que se centraliza el acceso a datos y simplifica el código de negocio. Además también facilita el cambio de fuente de datos. Como inconveniente se añade una capa más al diseño. (Oracle, s.f.-a) Este material es anterior al periodo de análisis (2021-2026) pero se incluye por ser el original (publicado por el propio fabricante de Java). Investigaciones recientes han estudiado estos tipos de clases de acceso a datos como parte diferenciadora de los sistemas intensivos en datos. Por ejemplo en (Muse et al., 2022) se ha analizado su evolución en 12

sistemas de código abierto. Se ha encontrado que son menos frecuentemente refactorizadas que otras clases.

Consultas Parametrizadas e Inyección SQL

La inyección SQL es un problema que surge cuando los datos introducidos por el usuario se concatena a la cadena de texto de la consulta. De esta forma los datos se ven como parte del código. En (Nasereddin et al., 2021) se hace una revisión de 60 trabajos relacionados con la detección y prevención de este tipo de ataque en aplicaciones web. Según la guía de prevención de OWASP el mecanismo fundamental de protección son las sentencias preparadas (con consultas parametrizadas). Esto consiste en separar el código SQL de los valores. Primero se envía el código SQL y luego los valores. De esta forma la base de datos siempre sabe qué partes son código y cuáles datos. (OWASP Cheat Sheet Series Team, s.f.) En (Yunanto y Al Ghifari, 2025) se compara un sistema de autenticación con vulnerabilidades (que usa concatenación de cadenas) con otro que usa sentencias preparadas. Se prueba con 15 cargas maliciosas. Todas fueron bloqueadas en la versión parametrizada. En la versión vulnerable fue superada en el 93.3% de los casos.

JDBC Frente a los Frameworks ORM

Los frameworks de mapeo objeto relacional (ORM) como Hibernate o EclipseLink generan automáticamente gran parte del SQL a partir de las clases Java. En (Żuchnik y Kopniak, 2021) se compara el uso de JDBC frente a distintos frameworks ORM. Se utiliza un conjunto de criterios ponderados por los programadores. En (Baran y Muryjas, 2023) se analiza el rendimiento de cuatro frameworks ORM en consultas analíticas. Hay notable diferencia entre ellos. En esta práctica se ha optado por el uso de JDBC directo. El objetivo pedagógico es que

entiendan cómo funciona el proceso de apertura de conexión, construcción de sentencias y recorrido de resultados. Todo esto es opaco en un ORM.

Descripción Sobre La Practica

El proyecto es un sistema de gestión de estudiantes. Es una aplicación de consola. Permite gestionar un registro de estudiantes. Los datos de un estudiante son documento de identidad, nombres, apellidos, correo electrónico, programa académico y semestre. Al arrancar la aplicación se crea (o abre) el fichero data/estudiantes.db. A continuación muestra un menú de opciones. Hay siete opciones: registrar, listar, buscar por ID, buscar por documento, actualizar, eliminar y salir. En el encabezado de menú aparece el número de estudiantes registrados. Así es fácil comprobar que los datos persisten entre ejecuciones.

Se ha optado por un dominio académico. Es un dominio conocido. Existen claras reglas de validación (formato de documento y correo, rango semestres). También es posible mostrar restricciones de integridad (unicidad documento y correo). La combinación Java 17 + Maven + SQLite es una buena opción por portabilidad. Se puede ejecutar en Windows, Linux o macOS. Con un único comando. No es necesario instalar y configurar un servidor de base de datos. El propio controlador contiene el motor de SQLite. (Xerial, s.f.) El uso de Maven y su estructura de carpetas estándar permite que cualquiera que conozca Maven pueda encontrar fácilmente el código fuente, recursos y pruebas. Sin necesidad de documentación adicional. (Apache Software Foundation, s.f.)

Requerimientos Funcionales y Técnicos

Requerimientos Funcionales

La Tabla 1 resume los requerimientos funcionales y la opción del menú o el mecanismo que los implementa.

Tabla 1

Requerimientos funcionales de la aplicación

Código	Requerimiento	Implementación
RF01	Registrar un estudiante con sus seis datos.	Opción 1; `EstudianteDAOImpl.insertar()`
RF02	Listar todos los estudiantes ordenados por apellidos.	Opción 2; `listarTodos()`
RF03	Buscar un estudiante por su ID.	Opción 3; `buscarPorId()`
RF04	Buscar un estudiante por su documento.	Opción 4; `buscarPorDocumento()`
RF05	Actualizar los datos de un estudiante, conservando los campos que se dejen vacíos.	Opción 5; `actualizar()`
RF06	Eliminar un estudiante previa confirmación.	Opción 6; `eliminar()`
RF07	Validar el formato y el rango de los datos ingresados.	`EstudianteService.validar()`
RF08	Impedir documentos o correos repetidos.	Restricciones `UNIQUE` y `DatoDuplicadoException`
RF09	Conservar los datos después de cerrar la aplicación.	Archivo SQLite `data/estudiantes.db`
RF10	Informar los errores sin terminar el programa.	Bloque `try-catch` en `MenuConsola.iniciar()`

Requerimientos Técnicos

Tabla 2

Requerimientos técnicos del proyecto

Código	Requerimiento técnico
RT01	Lenguaje Java, compilado para la versión 17 (`maven.compiler.release` = 17).
RT02	Acceso a datos mediante JDBC, usando exclusivamente `PreparedStatement` con parámetros.
RT03	Base de datos SQLite almacenada en un archivo local, sin servidor.

Código	Requerimiento técnico
RT04	Gestión de dependencias, compilación, pruebas y empaquetado con Apache Maven.
RT05	Arquitectura en capas con el patrón DAO (interfaz e implementación separadas).
RT06	Cierre automático de conexiones, sentencias y resultados con `try-with-resources`.
RT07	Creación automática del esquema al iniciar, sin borrar datos existentes.
RT08	Pruebas automatizadas con JUnit 5 sobre una base de datos temporal.
RT09	Ejecución en Windows, Linux y macOS mediante un JAR que incluye el controlador.
RT10	Código fuente versionado y publicado en un repositorio de GitHub.

Tecnologías Utilizadas

La Tabla 3 presenta las tecnologías empleadas, la versión declarada en el archivo `pom.xml` y la función de cada una en el proyecto.

Tabla 3

Tecnologías utilizadas

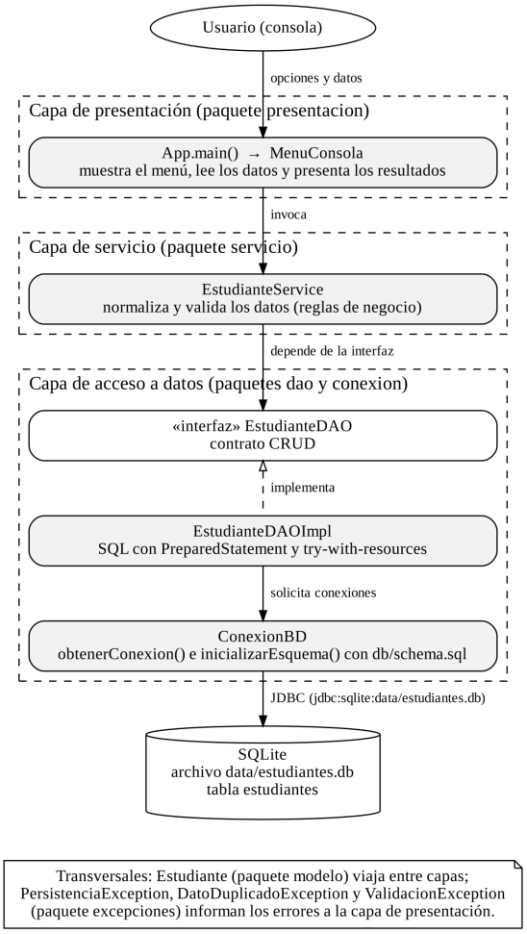
Tecnología	Versión	Función en el proyecto
Java (JDK)	17 o superior	Lenguaje de programación y máquina virtual.
JDBC (`java.sql`)	Incluida en el JDK	API estándar para conectarse a la base de datos y ejecutar SQL.
SQLite	Incluida en el controlador	Motor de base de datos embebido; almacena los datos en `data/estudiantes.db`.
sqlite-jdbc (`org.xerial`)	3.49.1.0	Controlador JDBC que implementa la API para SQLite.
Apache Maven	3.8 o superior	Gestión de dependencias, compilación, pruebas y generación del JAR.
maven-shade-plugin	3.6.0	Genera un JAR ejecutable que incluye el controlador.
exec-maven-plugin	3.5.0	Permite ejecutar la aplicación con `mvn exec:java`.
JUnit Jupiter	5.11.4	Marco de pruebas automatizadas.
Git y GitHub	No aplica	Control de versiones y publicación del repositorio.

Diseño de la Arquitectura

La aplicación se organiza en tres capas con responsabilidades separadas. La capa de presentación interactúa con el usuario; la capa de servicio aplica las reglas de negocio, y la capa de acceso a datos es la única que conoce JDBC y SQL. Cada capa depende solo de la inmediatamente inferior y, en el caso del acceso a datos, depende de la interfaz `EstudianteDAO` y no de su implementación, como propone el patrón DAO (Oracle, s.f.-a). La Figura 1 muestra esta organización.

Figura 1

Arquitectura en capas de la aplicación Gestión de Estudiantes



Nota. Elaboración propia a partir de las clases del proyecto.

Responsabilidad de Cada Clase

Tabla 4

Clases del proyecto y su responsabilidad

Clase	Paquete	Responsabilidad
`App`	raíz	Punto de entrada: inicializa el esquema, crea los objetos de cada capa e inicia el menú.
`MenuConsola`	`presentacion`	Muestra el menú, lee los datos con `Scanner` y presenta resultados y mensajes de error.
`EstudianteService`	`servicio`	Normaliza y valida los datos y delega las operaciones en el DAO.
`EstudianteDAO`	`dao`	Interfaz con el contrato de las operaciones CRUD.
`EstudianteDAOImpl`	`dao`	Implementa el CRUD con JDBC y `PreparedStatement`; traduce las `SQLException`.
`ConexionBD`	`conexion`	Construye la URL JDBC, abre conexiones y ejecuta `schema.sql`.
`Estudiante`	`modelo`	Entidad con los datos de un estudiante (una fila de la tabla).
`PersistenciaException`	`excepciones`	Error general de acceso a datos (no verificada).
`DatoDuplicadoException`	`excepciones`	Documento o correo repetido (violación de `UNIQUE`).
`ValidacionException`	`excepciones`	Dato que no cumple una regla de negocio.

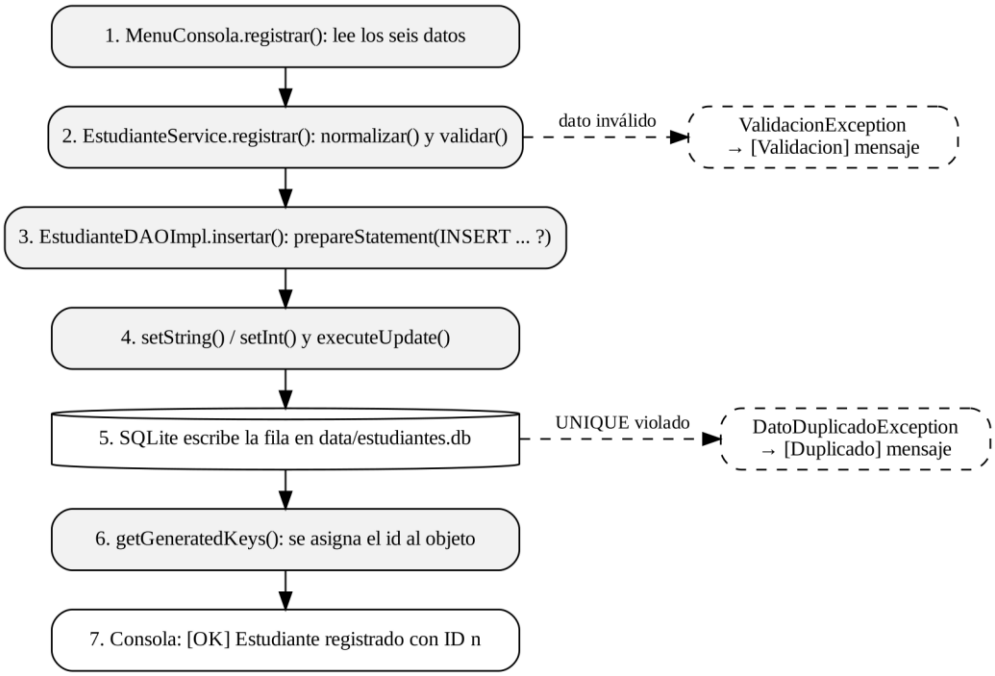
Nota. El paquete raíz es `co.edu.umb.persistencia`.

Flujo de Datos

La Figura 2 muestra el recorrido de los datos al registrar un estudiante. El menú lee los valores; el servicio elimina espacios sobrantes, pasa el correo a minúsculas y valida cada campo; el DAO prepara la sentencia `INSERT`, asigna los parámetros y la ejecuta, y finalmente obtiene el ID generado por la base de datos. Si una validación falla, la operación se detiene antes de llegar a la base de datos; si la base de datos rechaza un valor repetido, el error vuelve al menú como una `DatoDuplicadoException`.

Figura 2

Flujo de datos en la operación de registro



Nota. Elaboración propia. Las líneas discontinuas representan los caminos de error.

Diseño de la Base de Datos

La base de datos tiene una sola tabla, `estudiantes`, porque la práctica se centra en un único tipo de entidad. Las reglas de integridad se definen en la propia base de datos, de modo que se cumplen aunque otro programa modifique el archivo: el documento y el correo son únicos, todos los campos son obligatorios y el semestre debe estar entre 1 y 10. La Tabla 5 presenta el diccionario de datos y la Figura 3 el script que crea la tabla.

Tabla 5

Diccionario de datos de la tabla estudiantes

Columna	Tipo	Restricciones	Descripción
`id`	INTEGER	PRIMARY KEY AUTOINCREMENT	Identificador generado por la base de datos.
`documento`	TEXT	NOT NULL, UNIQUE	Documento de identidad (6 a 12 dígitos).
`nombres`	TEXT	NOT NULL	Nombres del estudiante.
`apellidos`	TEXT	NOT NULL	Apellidos; tienen un índice para ordenar el listado.
`correo`	TEXT	NOT NULL, UNIQUE	Correo electrónico en minúsculas.
`programa`	TEXT	NOT NULL	Programa académico.
`semestre`	INTEGER	NOT NULL, CHECK (1 a 10)	Semestre que cursa.
`fecha_registro`	TEXT	NOT NULL, DEFAULT fecha local	Fecha y hora de creación del registro.

Figura 3

Script de creación del esquema (src/main/resources/db/schema.sql)

```
-- Esquema de la base de datos SQLite de la aplicacion Gestion de Estudiantes.
-- Se ejecuta automaticamente al iniciar la aplicacion (clase ConexionBD).
-- "IF NOT EXISTS" garantiza que los datos existentes no se borren en cada inicio.

CREATE TABLE IF NOT EXISTS estudiantes (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    documento       TEXT    NOT NULL UNIQUE,
    nombres         TEXT    NOT NULL,
    apellidos       TEXT    NOT NULL,
    correo          TEXT    NOT NULL UNIQUE,
    programa        TEXT    NOT NULL,
    semestre        INTEGER NOT NULL CHECK (semestre BETWEEN 1 AND 10),
    fecha_registro  TEXT    NOT NULL DEFAULT (datetime('now', 'localtime'))
);

CREATE INDEX IF NOT EXISTS idx_estudiantes_apellidos ON estudiantes (apellidos);
```

Nota. Contenido literal del archivo del proyecto.

Inicialización y Ubicación del Archivo

El archivo de base de datos no se publica en el repositorio. Al iniciar, el método `inicializarEsquema()` crea la carpeta `data/` si no existe y ejecuta el script anterior; como todas las sentencias usan `IF NOT EXISTS`, los datos registrados en ejecuciones previas no se borran. Esta decisión evita publicar un archivo binario que cambia en cada ejecución y garantiza que el docente obtenga una base de datos nueva y válida al ejecutar el proyecto por primera vez. La ubicación puede cambiarse con la propiedad del sistema `db.path`.

Implementación de JDBC

Conexión con la Base de Datos

La clase `ConexionBD` recibe la ruta del archivo, la convierte en ruta absoluta y construye la URL JDBC con el formato `jdbc:sqlite:<ruta>`. El método `obtenerConexion()` usa `DriverManager.getConnection()`, que localiza automáticamente el controlador sqlite-jdbc declarado como dependencia en Maven. Si la conexión falla, la `SQLException` se envuelve en una `PersistenciaException` con un mensaje que incluye la URL (Figura 4).

Figura 4

Método obtenerConexion de la clase ConexionBD

```
public Connection obtenerConexion() {
    try {
        return DriverManager.getConnection(url);
    } catch (SQLException e) {
        throw new PersistenciaException("No fue posible conectar con la base de datos: " +
url, e);
    }
}
```

Nota. Código extraído del archivo `ConexionBD.java`.

Uso de PreparedStatement

Todas las sentencias SQL del proyecto se declaran como constantes con marcadores `?` y se ejecutan mediante `PreparedStatement`. Los valores se asignan con `setString()` y `setInt()`, de forma que nunca se concatenan al texto SQL. Esta es la defensa principal que recomienda OWASP contra la inyección SQL (OWASP Cheat Sheet Series Team, s.f.). La prueba `inyeccionSqlNoAfectaLaTabla` busca el documento `' OR '1'='1` y comprueba que no se devuelve ningún registro y que la tabla conserva su contenido.

Figura 5

Sentencias SQL parametrizadas de la clase EstudianteDAOImpl

```
private static final String SQL_INSERTAR =
        "INSERT INTO estudiantes (documento, nombres, apellidos, correo, programa, semestre) "
      + "VALUES (?, ?, ?, ?, ?, ?)";

private static final String SQL_COLUMNAS =
        "SELECT id, documento, nombres, apellidos, correo, programa, semestre, fecha_registro
"
      + "FROM estudiantes";

private static final String SQL_POR_ID = SQL_COLUMNAS + " WHERE id = ?";

private static final String SQL_POR_DOCUMENTO = SQL_COLUMNAS + " WHERE documento = ?";

private static final String SQL_LISTAR = SQL_COLUMNAS + " ORDER BY apellidos, nombres";

private static final String SQL_ACTUALIZAR =
        "UPDATE estudiantes SET documento = ?, nombres = ?, apellidos = ?, correo = ?, "
      + "programa = ?, semestre = ? WHERE id = ?";

private static final String SQL_ELIMINAR = "DELETE FROM estudiantes WHERE id = ?";

private static final String SQL_CONTAR = "SELECT COUNT(*) FROM estudiantes";
```

Nota. Código extraído del archivo `EstudianteDAOImpl.java`.

Gestión de Recursos

Cada método del DAO abre su propia conexión y la cierra al terminar mediante la instrucción `try-with-resources`, que cierra automáticamente `Connection`, `PreparedStatement` y `ResultSet` incluso si ocurre una excepción. Así se evita dejar conexiones abiertas o el archivo de SQLite bloqueado. Para una aplicación de consola de un solo usuario, abrir una conexión por operación es suficiente y mantiene el código simple.

Desarrollo de las Operaciones CRUD

La Tabla 6 relaciona cada operación CRUD con el método del DAO, la sentencia SQL y el método JDBC que la ejecuta.

Tabla 6

Correspondencia entre las operaciones CRUD, el DAO y JDBC

Operación	Método del DAO	Sentencia SQL	Método JDBC
Crear	`insertar(Estudiante)`	INSERT INTO estudiantes ... VALUES (?, ?, ?, ?, ?, ?)	`executeUpdate()` y `getGeneratedKeys()`
Leer	`buscarPorId(int)`	SELECT ... WHERE id = ?	`executeQuery()`
Leer	`buscarPorDocumento(String)`	SELECT ... WHERE documento = ?	`executeQuery()`
Leer	`listarTodos()`	SELECT ... ORDER BY apellidos, nombres	`executeQuery()`
Leer	`contar()`	SELECT COUNT(*) FROM estudiantes	`executeQuery()`
Actualizar	`actualizar(Estudiante)`	UPDATE estudiantes SET ... WHERE id = ?	`executeUpdate()`
Eliminar	`eliminar(int)`	DELETE FROM estudiantes WHERE id = ?	`executeUpdate()`

Crear

El método `insertar()` prepara la sentencia con la opción `Statement.RETURN_GENERATED_KEYS`, asigna los seis parámetros mediante el método auxiliar `asignarParametros()`, ejecuta la inserción y lee la clave generada para asignarla al objeto (Figura 6). De esta manera la aplicación puede informar el ID del nuevo registro.

Figura 6

Método insertar de la clase EstudianteDAOImpl

```
@Override
public Estudiante insertar(Estudiante estudiante) {
    try (Connection conexion = conexionBD.obtenerConexion();
         PreparedStatement ps = conexion.prepareStatement(SQL_INSERTAR,
Statement.RETURN_GENERATED_KEYS)) {

        asignarParametros(ps, estudiante);
        ps.executeUpdate();

        try (ResultSet claves = ps.getGeneratedKeys()) {
            if (claves.next()) {
                estudiante.setId(claves.getInt(1));
            }
        }
        return estudiante;
    } catch (SQLException e) {
        throw traducir("insertar el estudiante", e);
    }
}
```

Nota. Código extraído del archivo `EstudianteDAOImpl.java`.

Leer

Las búsquedas por ID y por documento devuelven un `Optional<Estudiante>`: vacío si no hay coincidencias y con el objeto si la fila existe. El listado recorre el `ResultSet` y convierte cada fila en un objeto mediante el método `mapear()`, que lee las columnas por su nombre.

Actualizar

Antes de actualizar, el menú busca el estudiante y muestra cada valor actual entre corchetes; si el usuario deja un campo vacío, se conserva el valor anterior. El servicio valida de nuevo el objeto completo y el DAO ejecuta la sentencia `UPDATE`, que reutiliza los seis parámetros de la inserción y añade el ID como séptimo parámetro. El método devuelve `true` solo si se modificó exactamente una fila.

Eliminar

La eliminación muestra el nombre del estudiante y pide confirmación con S o N, para evitar borrados accidentales. El método `eliminar()` devuelve `true` si se borró una fila; si el ID no existe, la aplicación lo informa sin generar un error.

Validaciones y Manejo de Excepciones

Reglas de Validación

Las validaciones se aplican en dos niveles. En la capa de servicio, `EstudianteService` revisa los datos con expresiones regulares y rangos antes de llegar a la base de datos (Tabla 7). En la base de datos, las restricciones `NOT NULL`, `UNIQUE` y `CHECK` actúan como una segunda barrera. Además, el menú convierte a número los valores de ID y semestre y rechaza los textos no numéricos.

Tabla 7

Reglas de validación implementadas en EstudianteService

Dato	Regla	Mensaje mostrado
Documento	Entre 6 y 12 dígitos	El documento debe tener entre 6 y 12 digitos.
Nombres y apellidos	Entre 2 y 60 letras, espacios, apóstrofos, puntos o guiones	Los nombres deben tener entre 2 y 60 letras.
Correo	Formato usuario@dominio.ext	El correo electronico no tiene un formato valido.
Programa	Entre 3 y 80 caracteres	El programa debe tener entre 3 y 80 caracteres.
Semestre	Número entero entre 1 y 10	El semestre debe estar entre 1 y 10.
ID	Número entero positivo	El id debe ser un numero entero positivo.

Nota. Antes de validar se eliminan los espacios sobrantes y el correo se convierte a minúsculas. Los mensajes se muestran sin tildes, tal como aparecen en la consola.

Jerarquía de Excepciones

El proyecto define tres excepciones no verificadas. `PersistenciaException` envuelve cualquier `SQLException`, de modo que las capas superiores no dependen de la API JDBC; `DatoDuplicadoException` la extiende para el caso particular de un documento o correo repetido, y `ValidacionException` representa los errores de reglas de negocio. El método `traducir()` del DAO examina el código de error de SQLite (19, `SQLITE_CONSTRAINT`) y el mensaje para identificar qué campo está duplicado (Figura 7). El menú captura cada tipo por separado y muestra un mensaje con el prefijo [Validacion], [Duplicado] o [Error de base de datos], tras lo cual vuelve a mostrar el menú.

Figura 7

Método traducir de la clase EstudianteDAOImpl

```
private PersistenciaException traducir(String operacion, SQLException e) {
    String mensaje = e.getMessage() == null ? "" : e.getMessage();
    if (e.getErrorCode() == SQLITE_CONSTRAINT && mensaje.contains("UNIQUE")) {
        String campo = mensaje.contains("estudiantes.correo") ? "correo" : "documento";
        return new DatoDuplicadoException("Ya existe un estudiante con ese " + campo + ".",
e);
    }
    return new PersistenciaException("Error al " + operacion + ": " + mensaje, e);
}
```

Nota. Código extraído del archivo `EstudianteDAOImpl.java`.

Pruebas Realizadas y Resultados Comprobados

Entorno de Pruebas

Las pruebas se ejecutaron el 21 de octubre de 2026 en el entorno de desarrollo (sistema Linux, OpenJDK 21.0.12, código compilado con la opción `--release 17`, controlador sqlite-jdbc 3.49.1.0). En ese entorno no fue posible descargar complementos desde el repositorio central de Maven; por ello, el código se compiló directamente con `javac`, las 16 clases de prueba se ejecutaron con un ejecutor equivalente y el JAR ejecutable se armó con las mismas

clases y el mismo controlador. La ejecución de `mvn clean package` en el equipo del autor queda pendiente de verificación y su resultado debe registrarse en este apartado antes de la entrega.

Pruebas Automatizadas

Las pruebas automatizadas usan un archivo SQLite temporal creado por JUnit, por lo que no modifican los datos reales. La Tabla 8 resume los resultados observados.

Tabla 8

Resultados de las pruebas automatizadas

N.º	Clase de prueba	Caso verificado	Resultado
1	EstudianteDAOImplTest	Actualizar: los cambios quedan guardados	Superada
2	EstudianteDAOImplTest	Leer: buscar un id inexistente devuelve Optional vacio	Superada
3	EstudianteDAOImplTest	Leer: buscar por id y por documento devuelve el registro guardado	Superada
4	EstudianteDAOImplTest	Restriccion UNIQUE: correo duplicado lanza DatoDuplicadoException	Superada
5	EstudianteDAOImplTest	Restriccion UNIQUE: documento duplicado lanza DatoDuplicadoException	Superada
6	EstudianteDAOImplTest	Eliminar: el registro deja de existir	Superada
7	EstudianteDAOImplTest	Crear: insertar asigna un id generado por la base de datos	Superada
8	EstudianteDAOImplTest	Inyeccion SQL: el texto malicioso se trata como dato, no como SQL	Superada
9	EstudianteDAOImplTest	Leer: listar devuelve todos los registros ordenados por apellidos	Superada
10	EstudianteDAOImplTest	Persistencia: los datos siguen disponibles al abrir una nueva conexion	Superada
11	EstudianteServiceTest	Correo sin arroba es rechazado	Superada
12	EstudianteServiceTest	Documento con letras es rechazado	Superada
13	EstudianteServiceTest	Un id negativo es rechazado antes de consultar la base de datos	Superada
14	EstudianteServiceTest	Nombres vacios son rechazados	Superada
15	EstudianteServiceTest	Un estudiante valido se registra y el correo se normaliza a minusculas	Superada

N.º	Clase de prueba	Caso verificado	Resultado
16	EstudianteServiceTest	Semestre fuera del rango 1-10 es rechazado	Superada

Nota. Resultado observado en el entorno de desarrollo: 16 pruebas ejecutadas, 16 superadas y 0 fallidas (Figura 15).

Pruebas Manuales de Funcionamiento

Además, se ejecutó el JAR en tres sesiones consecutivas sobre el mismo archivo de base de datos, alimentando el menú con los archivos de entradas guardados en `docs/evidencias/entradas-de-prueba`. La Tabla 9 compara el resultado esperado con el observado en cada caso.

Tabla 9

Casos de prueba manuales y resultados observados

N.º	Caso y datos de entrada	Resultado esperado	Resultado observado
1	Registrar tres estudiantes válidos	Mensaje de éxito con ID 1, 2 y 3	Cumple: IDs 1, 2 y 3
2	Listar estudiantes	Tres registros ordenados por apellidos	Cumple: Castro, Gómez, Martínez
3	Buscar por ID 2	Datos de Andrés Felipe Martínez López	Cumple
4	Buscar por documento 1034567890	Datos de María José Castro Díaz	Cumple
5	Actualizar ID 1: programa y semestre 4	Cambios guardados y visibles en el listado	Cumple
6	Registrar documento 1012345678 repetido	Mensaje [Duplicado]; no se inserta	Cumple
7	Registrar con correo «correo-invalido»	Mensaje [Validacion]	Cumple
8	Registrar con semestre 12	Mensaje [Validacion]	Cumple
9	Buscar ID «abc»	Mensaje de número no válido	Cumple
10	Buscar ID 99 inexistente	Mensaje de registro no encontrado	Cumple
11	Reiniciar y eliminar ID 3 con S	Registro eliminado; quedan 2	Cumple
12	Eliminar ID 2 y responder N	Eliminación cancelada	Cumple
13	Reiniciar la aplicación y listar	Se muestran los 2 registros guardados	Cumple

N.º	Caso y datos de entrada	Resultado esperado	Resultado observado
14	Leer el archivo .db sin la aplicación	La tabla contiene los mismos 2 registros	Cumple

Nota. Los casos 1 a 10 corresponden a la primera sesión, los casos 11 y 12 a la segunda y el caso 13 a la tercera. El caso 14 se hizo con el módulo sqlite3 de Python.

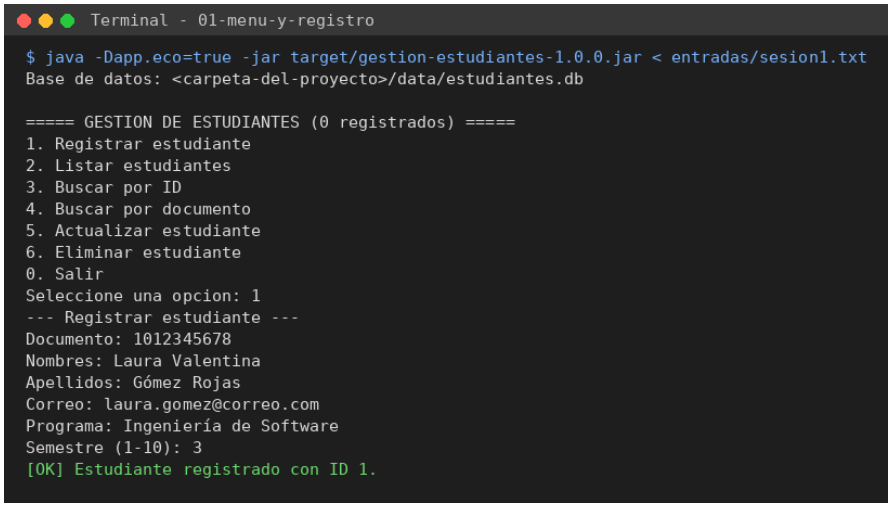
Los resultados confirman que las cuatro operaciones CRUD funcionan, que las validaciones detienen los datos incorrectos sin cerrar la aplicación y que la información persiste entre ejecuciones, porque la tercera sesión, iniciada como un proceso nuevo, mostró exactamente los registros que dejaron las sesiones anteriores.

Evidencias de Funcionamiento

Las Figuras 8 a 15 reproducen la salida real de la consola durante las sesiones descritas en la Tabla 9. Los archivos de texto y las imágenes se encuentran en la carpeta `docs/evidencias` del repositorio. La ruta absoluta del archivo de base de datos se reemplazó por `<carpeta-del-proyecto>` para no exponer rutas locales.

Figura 8

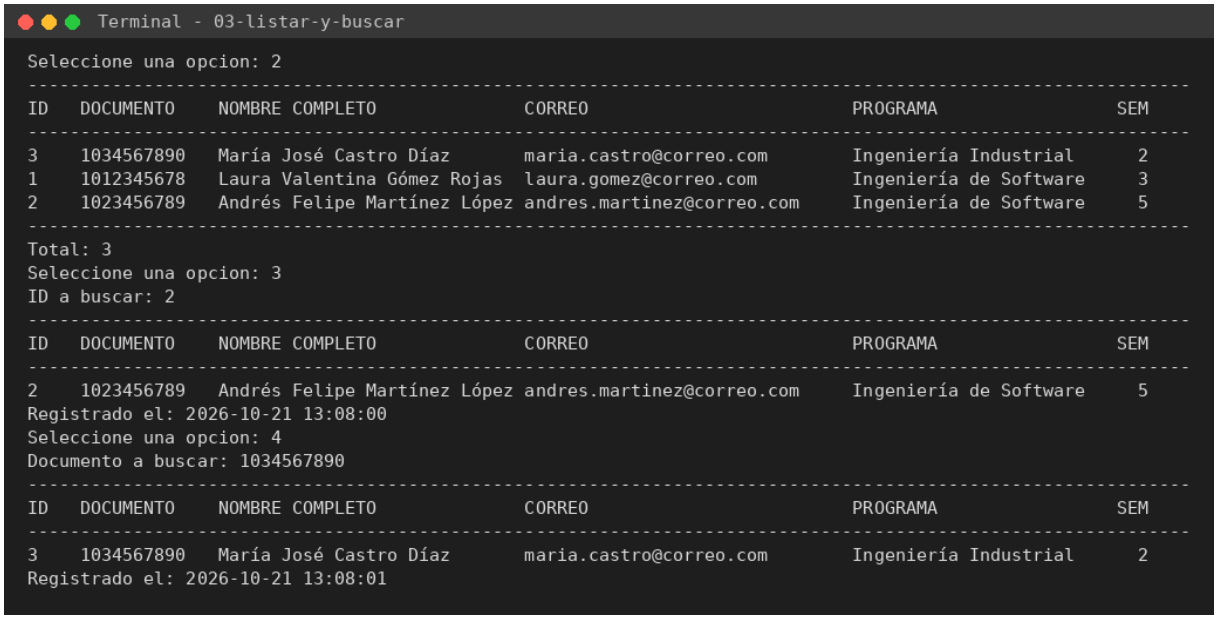
Menú principal y registro del primer estudiante



Nota. Salida real de la consola, registrada el 21 de octubre de 2026 en el entorno de desarrollo.

Figura 9

Listado y búsquedas por ID y por documento



Nota. Salida real de la consola, registrada el 21 de octubre de 2026 en el entorno de desarrollo.

Figura 10

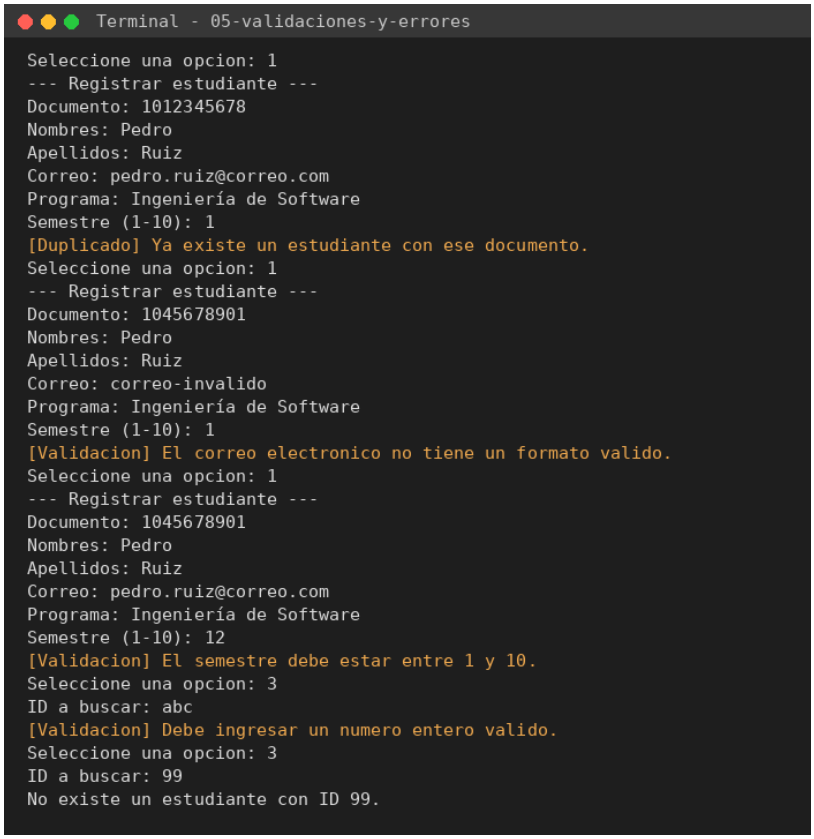
Actualización del programa y del semestre



Nota. Salida real de la consola, registrada el 21 de octubre de 2026 en el entorno de desarrollo.

Figura 11

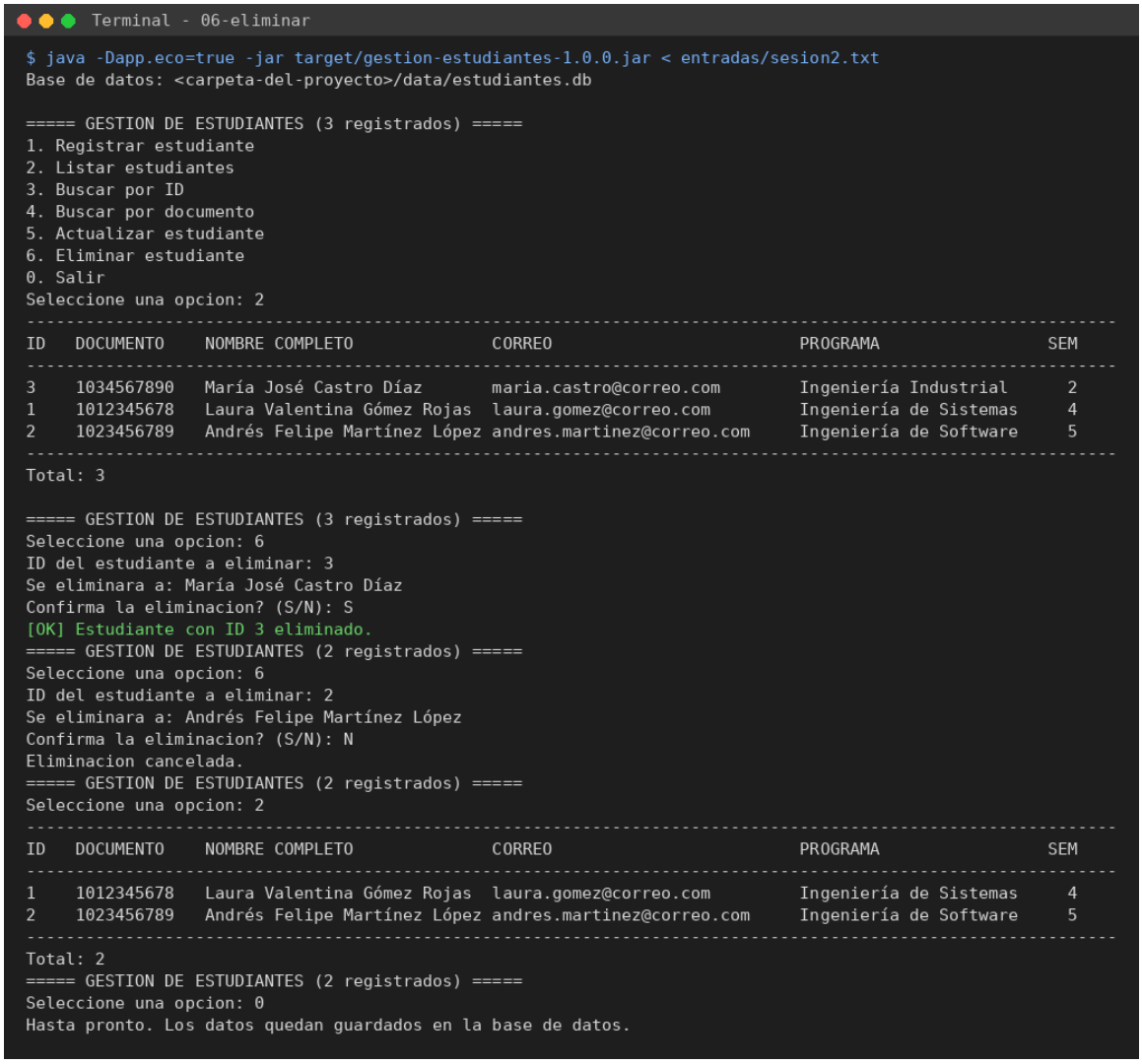
Validaciones y manejo de errores



Nota. Salida real de la consola, registrada el 21 de octubre de 2026 en el entorno de desarrollo.

Figura 12

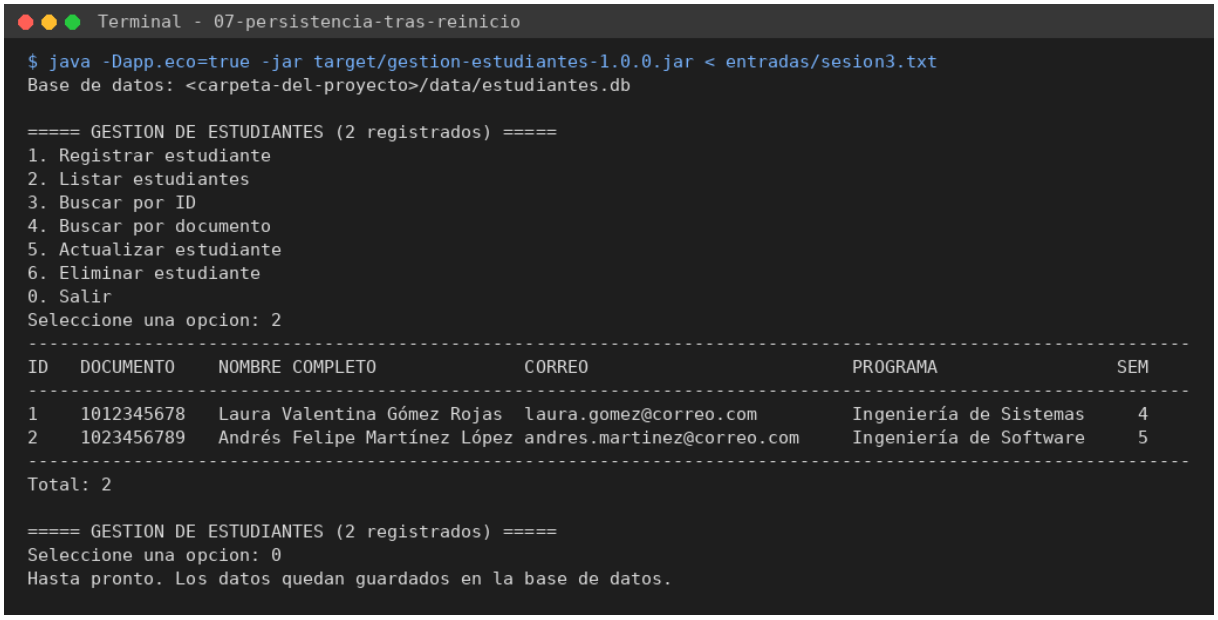
Eliminación confirmada y eliminación cancelada en la segunda sesión



Nota. Salida real de la consola, registrada el 21 de octubre de 2026 en el entorno de desarrollo.

Figura 13

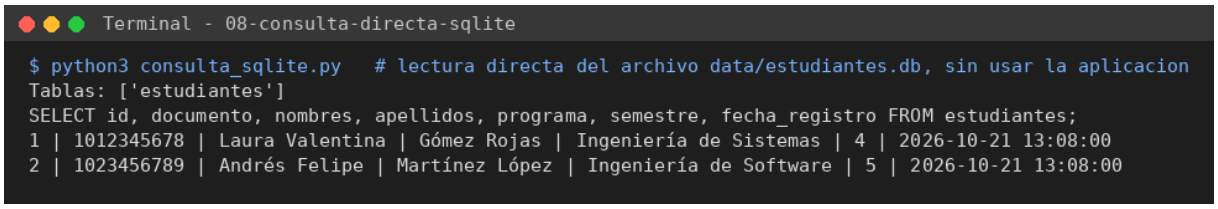
Datos disponibles después de reiniciar la aplicación (tercera sesión)



Nota. Salida real de la consola, registrada el 21 de octubre de 2026 en el entorno de desarrollo.

Figura 14

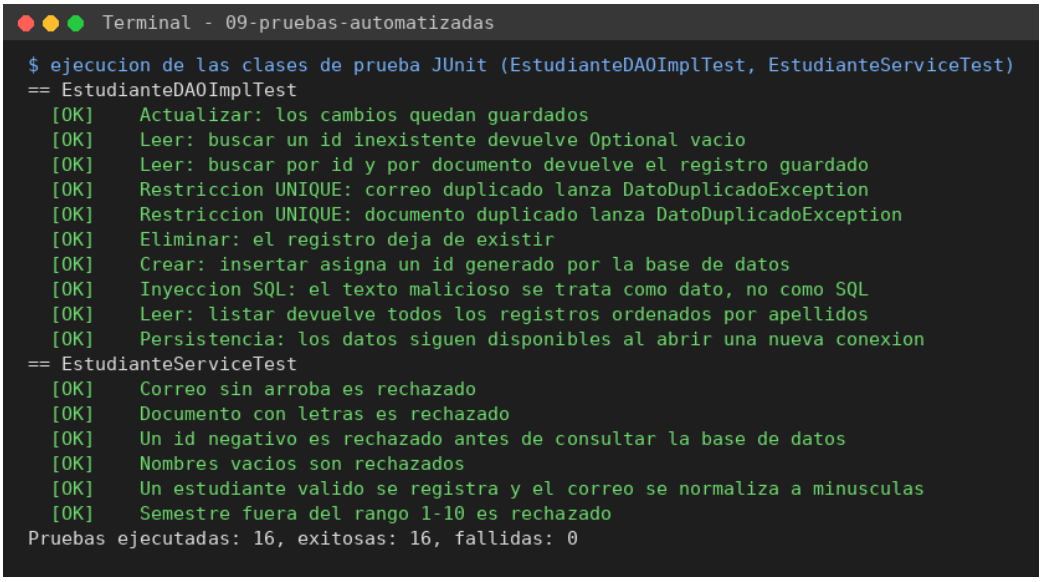
Consulta directa del archivo data/estudiantes.db sin usar la aplicación



Nota. Lectura del archivo con el módulo sqlite3 de Python, ejecutada después de la tercera sesión.

Figura 15

Resultado de las pruebas automatizadas



Nota. Salida real de la consola, registrada el 21 de octubre de 2026 en el entorno de desarrollo.

Instrucciones de Instalación y Ejecución

Para ejecutar el proyecto se necesita el JDK 17 o superior y Apache Maven 3.8 o superior; no es necesario instalar SQLite. Los pasos son los siguientes:

1. Verificar las herramientas con los comandos `java -version` y `mvn -version`.
2. Descargar el proyecto con `git clone https://github.com/[usuario]/taller-persistencia-datos-java.git` o con la opción Download ZIP de GitHub, y abrir una terminal en la carpeta del proyecto.
3. Compilar, ejecutar las pruebas y generar el JAR con `mvn clean package`. Al finalizar debe aparecer el mensaje BUILD SUCCESS y el archivo `target/gestion-estudiantes-1.0.0.jar`.

4. Ejecutar la aplicación con `java -jar target/gestion-estudiantes-1.0.0.jar`. Como alternativa, puede usarse `mvn compile exec:java` o ejecutar la clase `App` desde el IDE.
5. Registrar algunos estudiantes, salir con la opción 0 y volver a ejecutar el comando anterior para comprobar que los datos se conservan en `data/estudiantes.db`.

En Windows, si las tildes no se muestran correctamente en la consola, se recomienda ejecutar `chcp 65001` antes de iniciar la aplicación. El archivo `README.md` del repositorio contiene estas instrucciones y ejemplos de cada operación.

Enlaces de la Entrega

Repositorio de GitHub

El código fuente, el presente documento, el guion del video y las evidencias se encuentran en el repositorio: https://github.com/Kristianpng-90/persistencia-de-datos

Video de YouTube

El video explicativo, narrado en inglés, presenta la arquitectura, el código y una demostración de las operaciones y de la persistencia: [enlace pendiente de publicación].

Conclusiones

Se desarrolló una aplicación Java que gestiona estudiantes y conserva su información en una base de datos SQLite. Las sesiones de prueba demostraron que los datos registrados, modificados y eliminados se mantienen después de cerrar la aplicación, lo que cumple el objetivo central de la práctica.

JDBC ofrece un control explícito sobre cada conexión, sentencia y resultado. Aunque exige escribir más código que un framework ORM, este control permitió comprender el ciclo completo del acceso a datos, desde la URL de conexión hasta el mapeo de cada fila a un objeto.

El uso exclusivo de `PreparedStatement` separó el código SQL de los datos del usuario. La prueba con la cadena `' OR '1'='1` confirmó que el texto malicioso se trata como un valor común, lo que coincide con la recomendación de OWASP (OWASP Cheat Sheet Series Team, s.f.) y con los resultados de Yunanto y Al Ghifari (2025).

El patrón DAO y la arquitectura en capas aislaron el acceso a datos del resto del programa. El menú y el servicio no contienen SQL, de modo que cambiar SQLite por otro motor solo requeriría una nueva implementación de `EstudianteDAO`.

Combinar validaciones en el servicio con restricciones en la base de datos produjo una defensa en dos niveles: los errores comunes se detectan antes de llegar a SQLite y los casos que dependen de los datos almacenados, como los duplicados, se resuelven en la base de datos y se informan con mensajes claros. Como trabajo futuro, la aplicación podría ampliarse con una interfaz gráfica, más entidades relacionadas (por ejemplo, asignaturas y calificaciones) y un grupo de conexiones si se usa un motor cliente-servidor.

Referencias

Apache Software Foundation. (s.f.). *Introduction to the standard directory layout*. Apache Maven Project. Recuperado el 21 de octubre de 2026 de https://maven.apache.org/guides/introduction/introduction-to-the-standard-directory-layout.html

Baran, J., y Muryjas, P. (2023). The analysis of Java ORM frameworks performance in terms of analytical data processing. *Journal of Computer Sciences Institute*, *27*, 178–185. https://doi.org/10.35784/jcsi.3632

Gaffney, K. P., Prammer, M., Brasfield, L., Hipp, D. R., Kennedy, D., y Patel, J. M. (2022). SQLite: Past, present, and future. *Proceedings of the VLDB Endowment*, *15*(12), 3535–3547. https://doi.org/10.14778/3554821.3554842

Liu, W., Mondal, S., y Chen, T.-H. (2024). An empirical study on the characteristics of database access bugs in Java applications. *ACM Transactions on Software Engineering and Methodology*, *33*(7), 1–25. https://doi.org/10.1145/3672449

Muse, B. A., Khomh, F., y Antoniol, G. (2022). Do developers refactor data access code? An empirical study. En *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)* (pp. 25–35). IEEE. https://doi.org/10.1109/SANER53432.2022.00014

Nasereddin, M., ALKhamaiseh, A., Qasaimeh, M., y Al-Qassas, R. (2021). A systematic review of detection and prevention techniques of SQL injection attacks. *Information Security Journal: A Global Perspective*. Publicación anticipada en línea. https://doi.org/10.1080/19393555.2021.1995537

Oracle. (s.f.-a). *Core J2EE patterns: Data access object*. Recuperado el 21 de octubre de 2026 de https://www.oracle.com/java/technologies/dataaccessobject.html

Oracle. (s.f.-b). *Lesson: JDBC basics*. The Java Tutorials. Recuperado el 21 de octubre de 2026 de https://docs.oracle.com/javase/tutorial/jdbc/basics/

OWASP Cheat Sheet Series Team. (s.f.). *SQL injection prevention cheat sheet*. OWASP Foundation. Recuperado el 21 de octubre de 2026 de https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

SQLite. (2025, 31 de mayo). *Appropriate uses for SQLite*. https://sqlite.org/whentouse.html

Xerial. (s.f.). *SQLite JDBC driver* [Repositorio de software]. GitHub. Recuperado el 21 de octubre de 2026 de https://github.com/xerial/sqlite-jdbc

Yunanto, A. A., y Al Ghifari, M. G. (2025). Analisis efektivitas parameterized queries dalam pencegahan serangan SQL injection. *Jurnal Informatika Polinema*, *12*(1), 53–62. https://doi.org/10.33795/jip.v12i1.8977

Żuchnik, M., y Kopniak, P. (2021). Comparative analysis of connection performance with databases via JDBC interface and ORM programming frameworks. *Journal of Computer Sciences Institute*, *21*, 309–315. https://doi.org/10.35784/jcsi.2729